

INDIAN INSTITUTE OF TECHNOLOGY JODHPUR

# Campus Navigation System

An Application of Data Structures and Shortest Path Algorithms

## Project Report

Submitted as part of the course requirements for **Data Structures and Algorithms (DSA)**.

**Course:** Data Structures and Algorithms (CS/CM)

**Group Members:** **Pranshu Desai** (B24CS1057)  
**Shardul Diwate** (B24CS1028)  
**Vansh Rathi** (B24CM1065)  
**Katravath Kurumurthy** (B24CM1035)

# 1. Introduction

---

Efficient spatial traversal within modern university campuses is essential given their massive geographic expansion and infrastructural growth. The **IIT Jodhpur Campus Navigation System** is a targeted Data Structures and Algorithms software application designed to compute and display the optimal, minimum-distance walking paths between major academic, residential, and recreational buildings on campus.

By mapping key geographic milestones as mathematical components, this project models the IIT Jodhpur campus as a **weighted, undirected graph**. To satisfy distinct software application paradigms, the system is engineered across two independent, fully decoupled implementations: a low-overhead, high-performance command-line engine built from scratch in **C++17**, and an interactive graphical visualizer written in **Python 3** using network modeling and plotting libraries.

## 2. Problem Definition & Objectives

---

The objective of this project is to build an internal campus navigation solver capable of resolving a common logistical challenge: finding the fastest or shortest path between two points within a complex, interconnected layout.

### Primary Technical Objectives:

- Represent the structural framework of the IITJ campus landscape using memory-efficient data structures.
- Manually build and apply **Dijkstra's Shortest Path Algorithm** to traverse graph infrastructure with optimal runtime constraints.
- Expose precise metrics, yielding absolute walking paths (ordered sequencing of nodes) alongside estimated travel periods measured in minutes.
- Deliver multi-platform interfaces, offering both text-based terminal utilities (with embedded ASCII illustrations) and rich analytical graph plots.

## 3. Data Structures & Algorithmic Concepts

---

The underlying system architectural design draws extensively on foundational data structures to optimize performance, access overhead, and traversal complexity:

### | 3.1 Graph Representation

The campus layout is modeled explicitly as a graph  $G = (V, E)$ , where:

- **Vertices (V):** Represent structural hubs, complexes, or intersections (e.g., Hostel B1, Lecture Hall Complex 1, Main Gate, Library).

- **Edges (E):** Represent walkable paths directly connecting these landmarks.
- **Weights (W):** Non-negative real numbers mapped to edges representing the approximate time (in minutes) required to walk between the connected nodes.

The program stores the graph via an **Adjacency List** mapped through hash tables (*unordered\_map* in C++). This ensures an optimal spatial footprint of  $O(V + E)$ , which is significantly more efficient for sparse campus maps than a dense  $O(V^2)$  adjacency matrix.

### | 3.2 Priority Queue (Min-Heap)

To avoid the expensive  $O(V)$  scan for discovering unvisited nodes with minimal tentative distances, the program integrates a **binary min-heap** structure (*std::priority\_queue* in C++). It maintains nodes sorted by their current shortest estimated distance, compressing the lookups to a rapid  $O(1)$  time profile.

### | 3.3 Dijkstra's Algorithm and Time Complexity

Dijkstra's greedy optimization strategy ensures that when a vertex is extracted from the min-heap, its shortest path from the source node is definitively locked. The state transition updates tentative paths via edge relaxation:

$$\text{if } (\text{dist}[u] + \text{weight}(u, v) < \text{dist}[v]) \rightarrow \text{dist}[v] = \text{dist}[u] + \text{weight}(u, v)$$

By prioritizing extractions through the min-heap and managing direct structural updates across adjacency elements, the algorithmic architecture runs with a strict asymptotic upper bound of:

$$O(E \cdot \log V)$$

Where  $V$  is the count of operational nodes and  $E$  is the total number of bidirectional links on campus.

## 4. System Implementation Overview

---

### | 4.1 Implementation 1: C++ Core Logic Engine

The C++ version is optimized for performance and zero external dependencies, utilizing raw Standard Template Library (STL) components. It structures node maps manually, handles terminal reading, isolates lookup maps, and formats an ASCII topological map layout directly into the shell prompt.

Below is the core programmatic engine block executing the shortest path algorithm logic:

```

void dijkstra(const string &source, const string &destination) {
    if (!adj.count(source) || !adj.count(destination)) {
        cout << "
Error: One or both locations do not exist.
";
        return;
    }

    unordered_map<string, double> dist;
    unordered_map<string, string> parent;

    for (auto &p : adj) dist[p.first] = DBL_MAX;
    dist[source] = 0;

    using P = pair<double, string>;
    priority_queue<P, vector<P>, greater<P>> pq;
    pq.push({0.0, source});

    while (!pq.empty()) {
        auto top = pq.top();
        pq.pop();

        double currDist = top.first;
        string currNode = top.second;

        if (currDist > dist[currNode]) continue;

        for (auto &edge : adj[currNode]) {
            string next = edge.first;
            double weight = edge.second;

            if (dist[currNode] + weight < dist[next]) {
                dist[next] = dist[currNode] + weight;
                parent[next] = currNode;
                pq.push({dist[next], next});
            }
        }
    }
    // Path extraction and display block follows...
}

```

## 4.2 Implementation 2: Python Graphical Visualizer

The Python implementation targets high-level mathematical modeling and clear physical representation. It relies on the `networkx` library to manage the operational graph definitions and `matplotlib.pyplot` to render the map coordinates visually.

A key highlight of the Python script is its spatial placement routine. It maps exact 2D coordinate pairs ( $x, y$ ) onto nodes to simulate an organic, proportional layout of the campus grounds. Once the optimal trajectory is calculated, the system re-renders the selected nodes and transitions dynamically, redrawing the relevant edges with a prominent red highlight.

## 5. Execution, Testing, and Verification

Rigorous verification tests were performed to confirm the operational accuracy of the system. Both code implementations were evaluated using matching edge-case parameters, and they generated identical results.

Test Case Scenario:

Source Node Input: B1 | Target Destination Input: Sports Complex

Computed Shortest Distance: 13 minutes

Calculated Sequence Trajectory: B1 → G2 → Veg Mess → Sports Complex

During execution, the C++ variant outputs an orderly sequence to the console interface, while the Python pipeline displays a matching text report along with a standalone GUI window. In this visual window, the paths connecting the nodes B1, G2, Veg Mess, and Sports Complex are highlighted in red, providing a clear and immediate validation of the system's correctness.

## 6. Team Member Contribution Matrix

The implementation workload of this project was divided equitably among all group members, allowing each to focus on specific architectural modules:

| Group Member         | Roll Number | Core Project Contributions                                                                                                                                                                                                                         |
|----------------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pranshu Desai        | B24CS1057   | Designed the comprehensive campus graph structure. Mapped physical buildings to mathematical node structures, compiled accurate weight constants, built the initial C++ Graph skeleton class, and configured the structural adjacency list models. |
| Shardul Diwate       | B24CS1028   | Programmed the custom Dijkstra algorithm engine in C++. Developed the low-level min-heap priority queue logic, distance/parent hash tables, path backtracking algorithms, and managed boundary condition testing.                                  |
| Vansh Rathi          | B24CM1065   | Developed the Python graphical visualizer. Formatted spatial positions utilizing 2D coordinate arrays, configured the NetworkX parsing loops, and established the customized Matplotlib rendering styles to highlight active paths.                |
| Katravath Kurumurthy | B24CM1035   | Led quality assurance, testing pipelines, and manual calculation cross-checks. Structured the definitive presentation sequences, engineered project documentation workflows, and drafted the project development timeline reports.                 |

## 7. Conclusion and Future Enhancements

---

The Campus Navigation System successfully provides an efficient and mathematically verified approach to finding the shortest path across the IIT Jodhpur campus layout. By leveraging adjacency lists and min-heap priority queues, the system keeps its runtime complexity within a highly efficient  $O(E \cdot \log V)$  profile, ensuring immediate results.

### Proposed Future Work:

- **Real-World Map Integration:** Upgrade from a stylized scatter layout to a functional Graphical User Interface (GUI) layered directly on top of an accurate satellite map of IIT Jodhpur.
- **Real-Time GPS Tracking:** Integrate live geolocation feeds to track user movement dynamically and adjust paths relative to real-time positions.
- **Algorithmic Evolution:** Transition from Dijkstra's algorithm to an **A\* Search Algorithm**, utilizing Euclidean distance heuristics to minimize unnecessary node evaluation and optimize computing speed.